

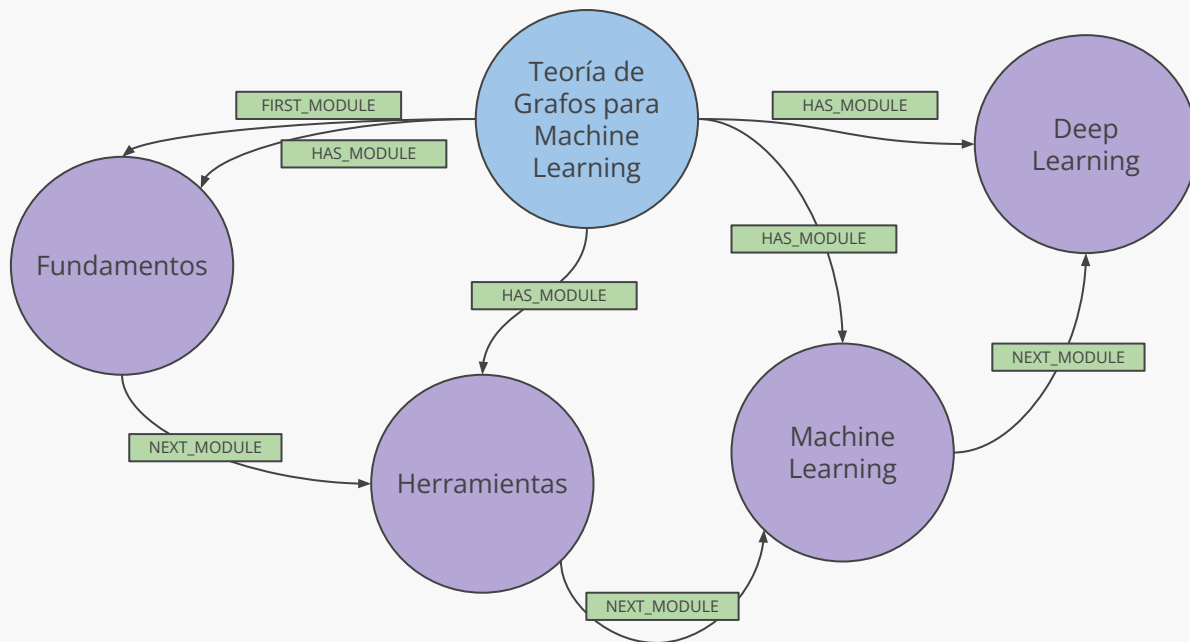
Clase 2

Algoritmos y analítica

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Contenidos del curso

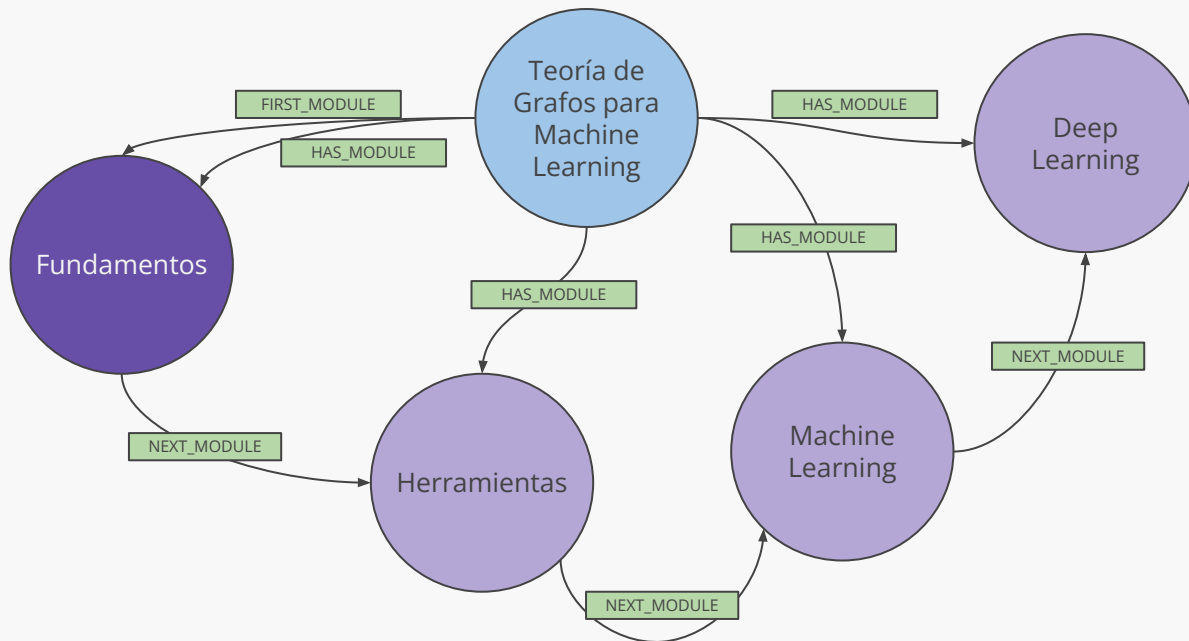


Contenidos del curso

1. Fundamentos

Desde conceptos a **algoritmos fundamentales** en teoría de grafos...

... para establecer una base sólida en los conceptos fundamentales de la teoría de grafos

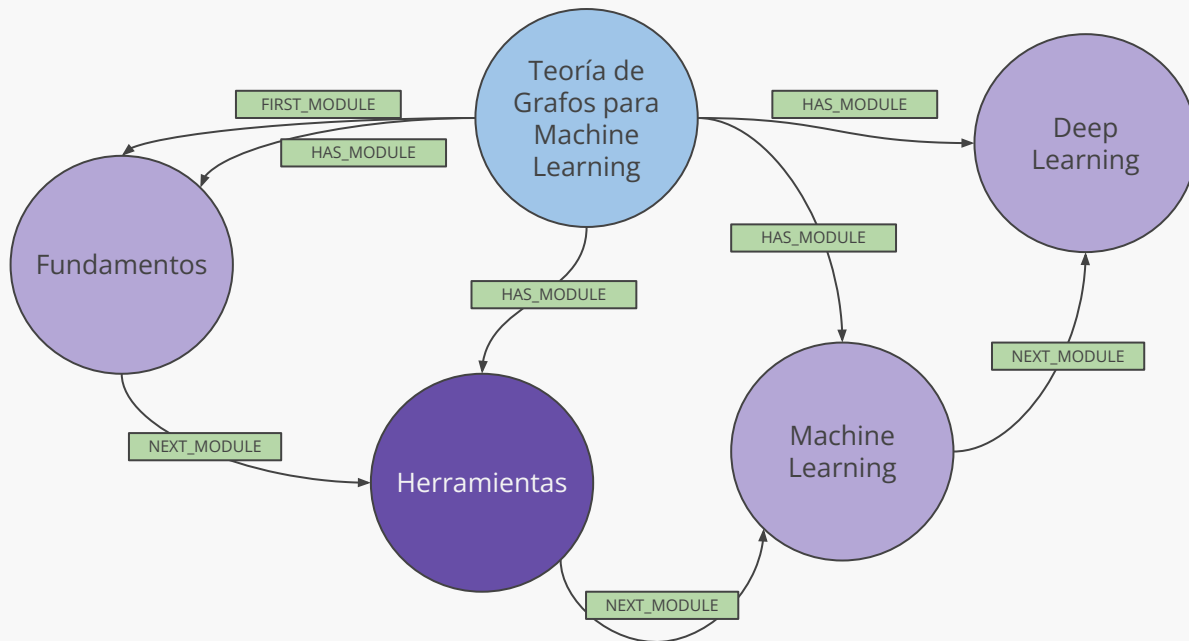


Contenidos del curso

2. Herramientas

Desde **representación de pequeños grafos** a analítica de grandes redes...

... para explorar y utilizar algoritmos fundamentales sobre grafos de cualquier tamaño en distintos contextos



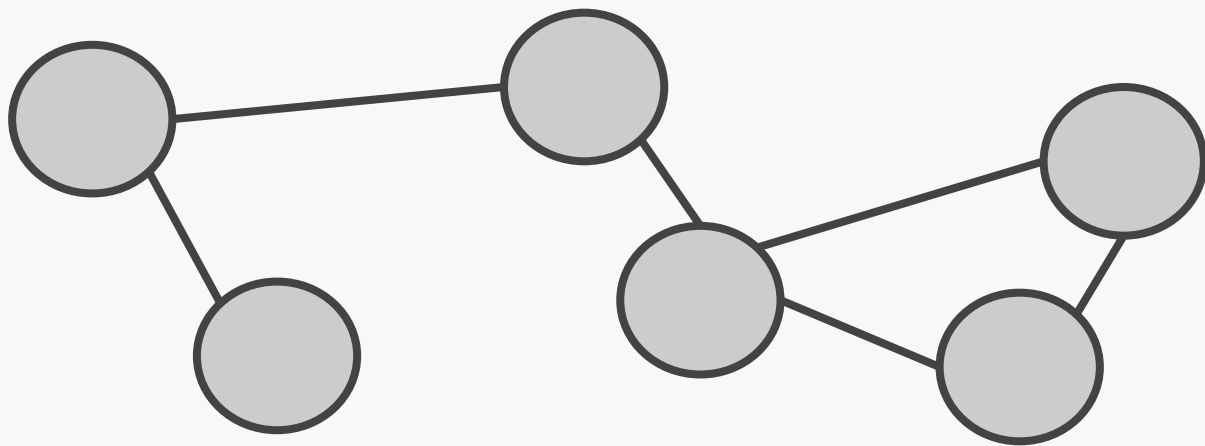
Clase 2.1

NetworkX

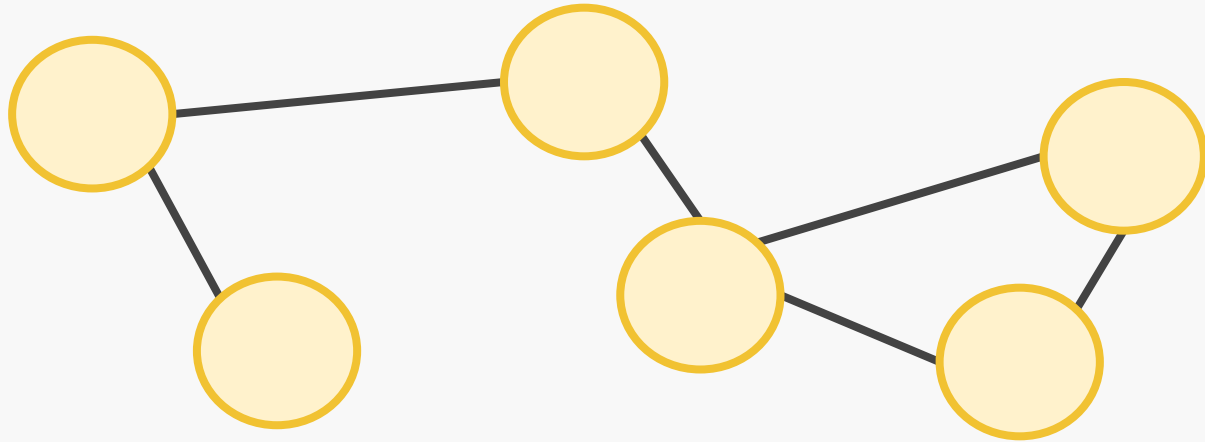
Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

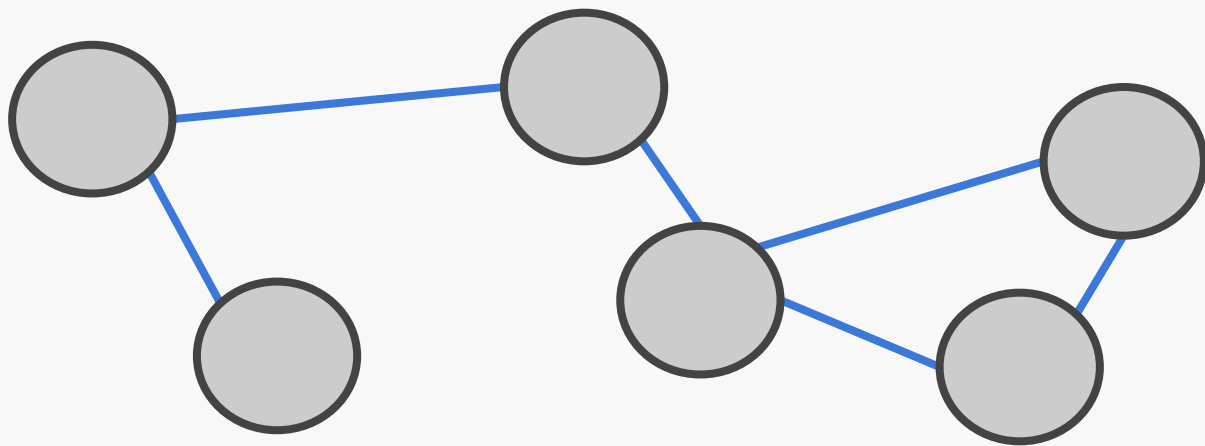
Grafo



Nodos o vértices



Arcos o aristas



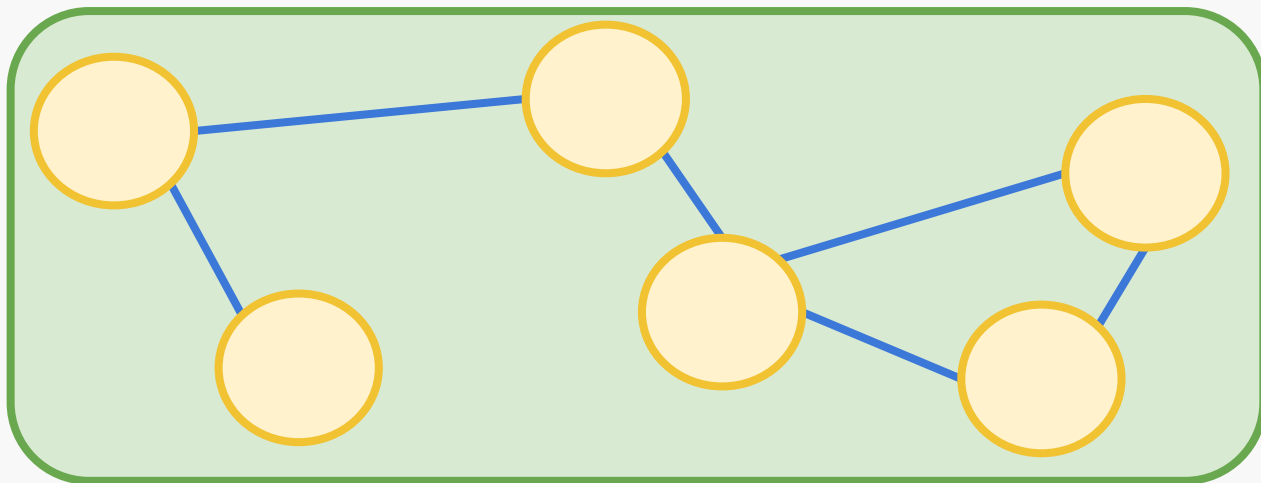
Componentes de una red

- **Objetos**: nodos, vértices (N)
- **Interacciones**: arcos, aristas
- **Sistema**: red, grafo

N (conjunto de nodos)

E (conjunto de aristas)

$G(N, E)$ (grafo)



Un grafo puede representar cualquier cosa

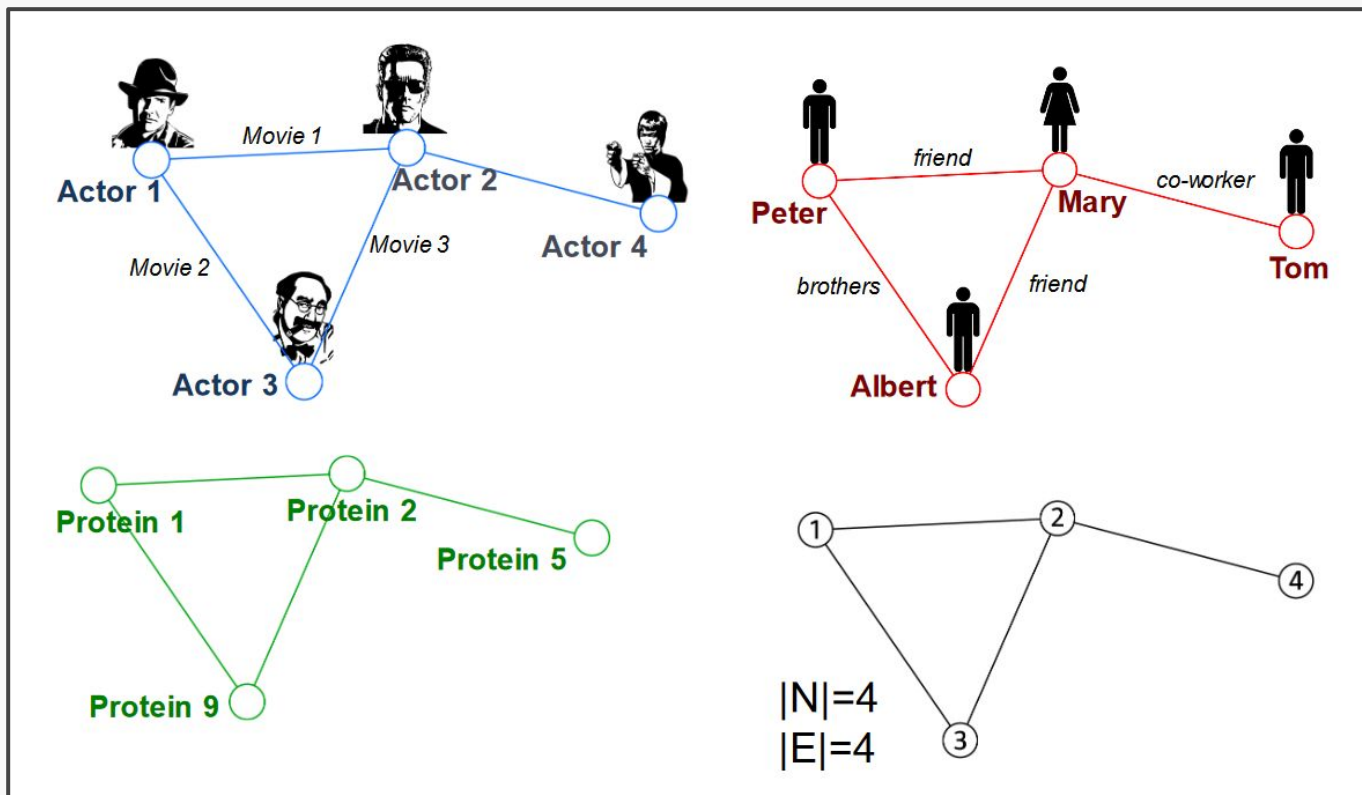
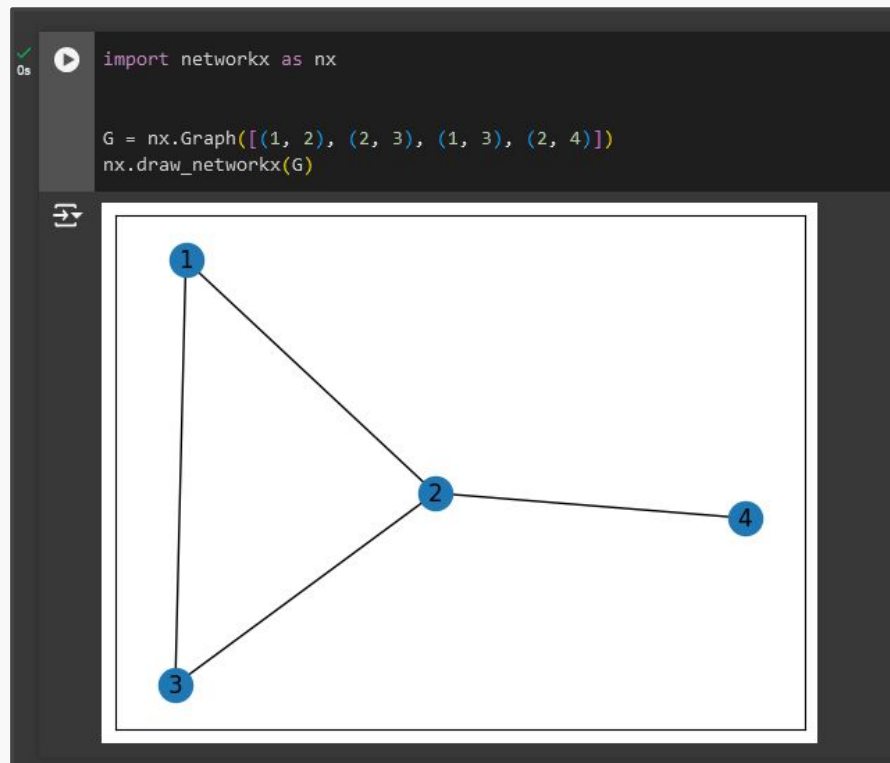
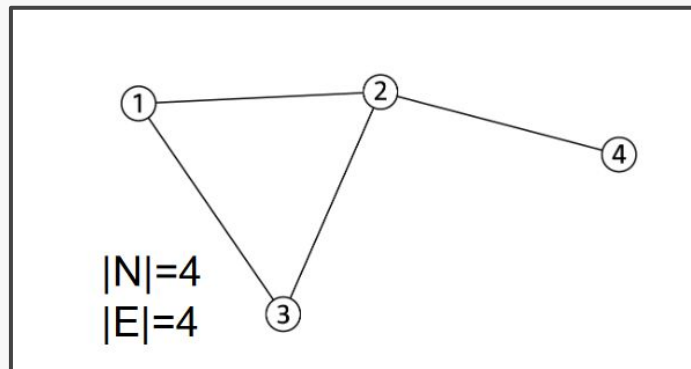


Figura: [CS224W: Machine Learning with Graphs \(Iure Leskovec\) @ Stanford](#)

Un grafo puede representar cualquier cosa



Contact

[Mailing list](#)

[Issue tracker](#)

[Source](#)

Releases

Stable ([notes](#))

3.3 — April 2024

[Documentation](#)

Latest ([notes](#))

3.4 development

[Documentation](#)

[Archive](#)



NetworkX

Network Analysis in Python

NetworkX is a Python package for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks.

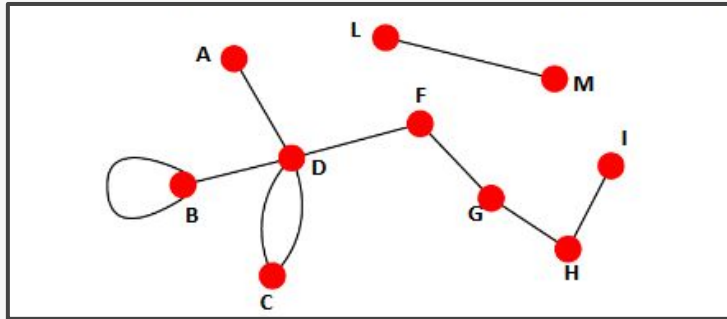
Software for complex networks

- Data structures for graphs, digraphs, and multigraphs
- Many standard graph algorithms
- Network structure and analysis measures
- Generators for classic graphs, random graphs, and synthetic networks
- Nodes can be "anything" (e.g., text, images, XML records)
- Edges can hold arbitrary data (e.g., weights, time-series)
- Open source [3-clause BSD license](#)
- Well tested with over 90% code coverage
- Additional benefits from Python include fast prototyping, easy to teach, and multi-platform

Grafos no dirigidos vs Grafos dirigidos

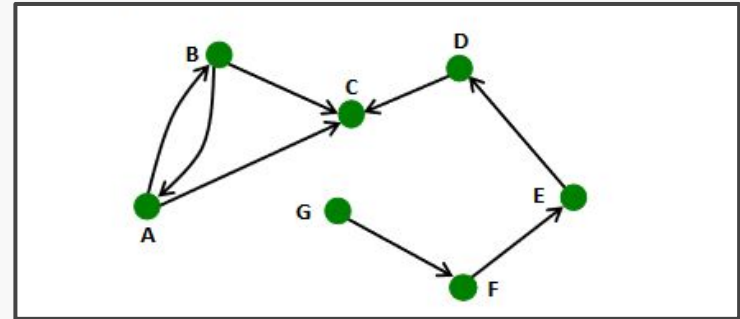
No dirigidos

- Arcos sin dirección (simétricos)
- Ejemplos: amistad en Facebook



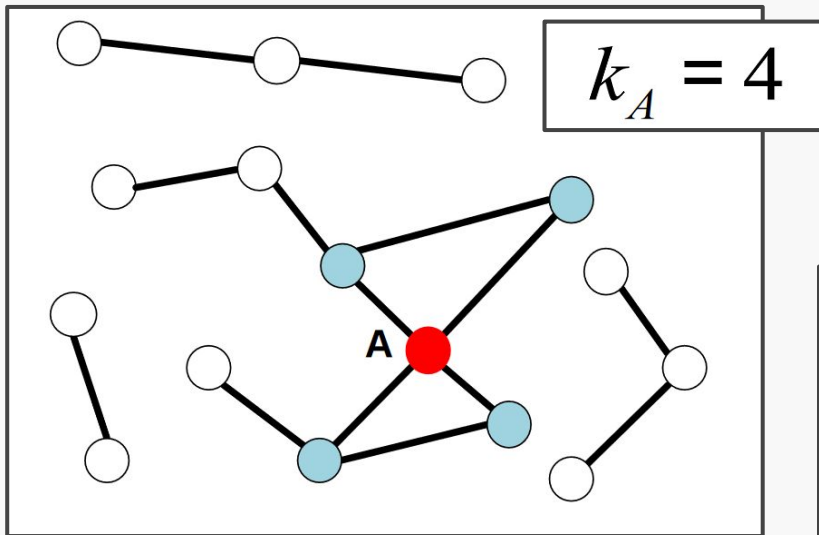
Dirigidos

- Arcos tienen sentido
- Ejemplos: seguir en Twitter (X), citaciones de papers



Grado de un nodo

El **grado de un vértice/nodo** es el número de aristas incidentes al vértice, básicamente, el **número de arcos que se conectan a un nodo particular**:

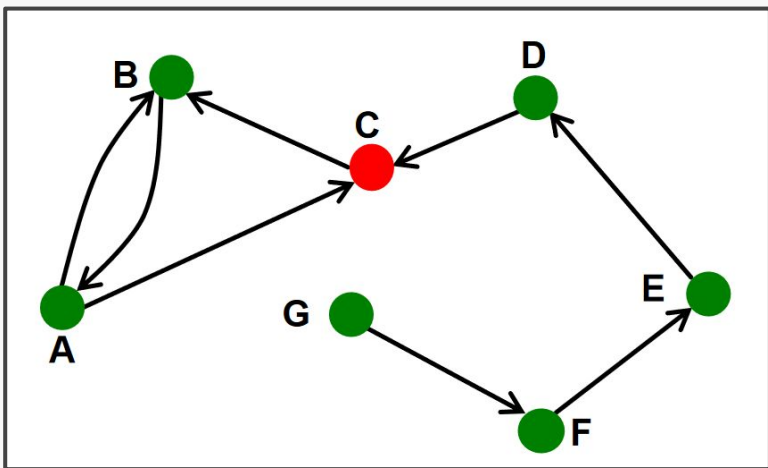


Grado promedio:

$$\bar{k} = \langle k \rangle = \frac{1}{N} \sum_{i=1}^N k_i = \frac{2E}{N}$$

Grado de un nodo

En el caso de grafos dirigidos, definimos un **grado de entrada** (in-degree) y un **grado de salida** (out-degree) según la cantidad de arcos que llegan o parten desde el nodo de interés



$$k_C^{in} = 2 \quad k_C^{out} = 1 \quad k_C = 3$$

$$\bar{k} = \frac{E}{N}$$

Grado
promedio

$$\overline{k^{in}} = \overline{k^{out}}$$

Grafos heterogéneos

Un grafo heterogéneo se puede definir como $G = (V, E, R, T)$, donde:

- V es el conjunto de nodos
- E son las relaciones entre nodos: (v_1, r, v_2)
- T indica el tipo de nodo: $T(v)$
- R es el conjunto de tipos de relaciones (r es parte de R)

Grafos heterogéneos

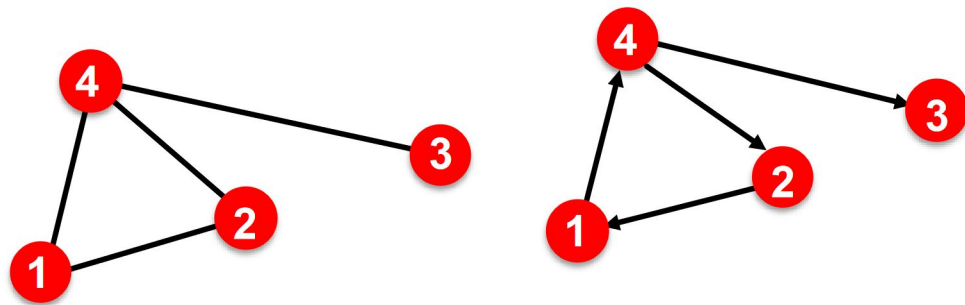
Un grafo heterogéneo se puede definir como $G = (V, E, R, T)$, donde:

- V es el conjunto de nodos
- E son las relaciones entre nodos: $(v1, r, v2)$

No soportado por NetworkX (directamente)

Matriz de adyacencia

Dado un grafo, podemos representarlo con una matriz indicando la existencia de arcos entre nodos

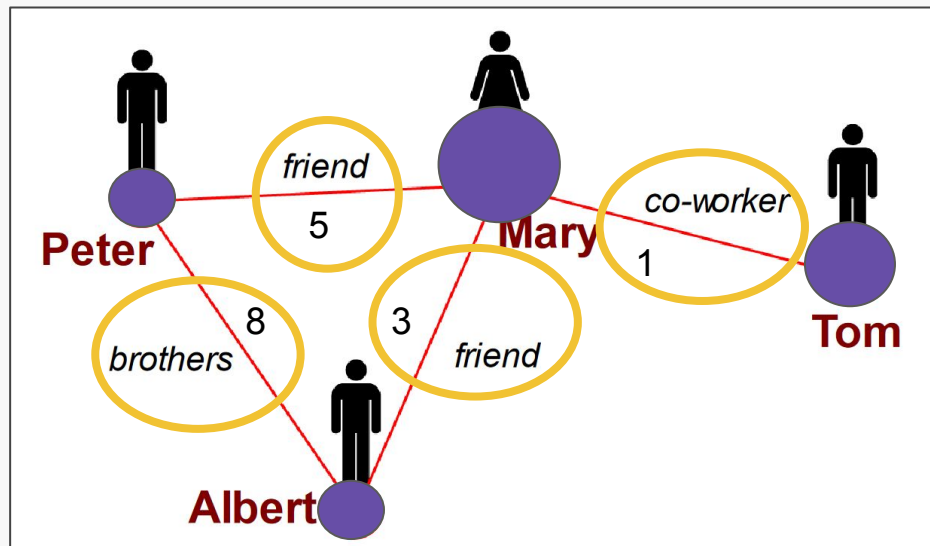


$A_{ij} = 1$ si existe un arco de i a j
 $A_{ij} = 0$ en caso contrario

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \quad A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Atributos de nodos y arcos

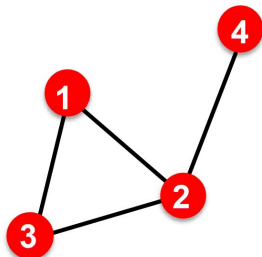
- Pesos
ej: número de interacciones
- Ranking
mayor a menor importancia
- Tipo
ej: amigo, familiar, compañero, etc.
- Propiedades dependiendo de la estructura del resto del grafo
ej: número de amigos en común



Más tipos de grafos

Grafo sin pesos (no dirigido)

Ej: amistad, *links*
entre sitios web



$$A_{ij} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

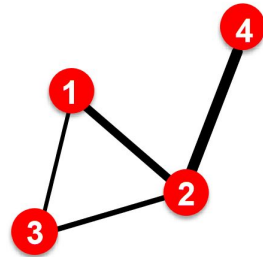
$$A_{ii} = 0$$

$$A_{ij} = A_{ji}$$

$$E = \frac{1}{2} \sum_{i,j=1}^N A_{ij} \quad \bar{k} = \frac{2E}{N}$$

Grafo con pesos (no dirigido)

Ej: velocidad
internet,
cantidad de
interacciones



$$A_{ij} = \begin{pmatrix} 0 & 2 & 0.5 & 0 \\ 2 & 0 & 1 & 4 \\ 0.5 & 1 & 0 & 0 \\ 0 & 4 & 0 & 0 \end{pmatrix}$$

$$A_{ii} = 0$$

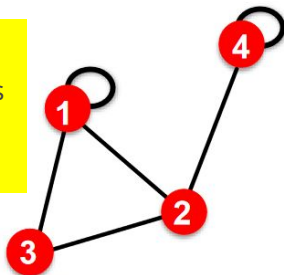
$$A_{ij} = A_{ji}$$

$$E = \frac{1}{2} \sum_{i,j=1}^N \text{nonzero}(A_{ij}) \quad \bar{k} = \frac{2E}{N}$$

Más tipos de grafos

Grafo con bucles (self-loop)

Ej: proteínas,
links entre sitios
web (auto
referencia)

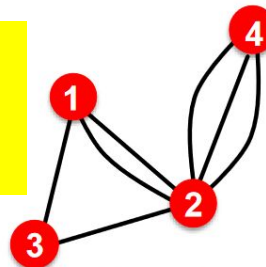


$$A_{ij} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

$$A_{ii} \neq 0 \quad A_{ij} = A_{ji}$$
$$E = \frac{1}{2} \sum_{i,j=1, i \neq j}^N A_{ij} + \sum_{i=1}^N A_{ii}$$

Multigrafo

Ej:
comunicación,
colaboración



$$A_{ij} = \begin{pmatrix} 0 & 2 & 1 & 0 \\ 2 & 0 & 1 & 3 \\ 1 & 1 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{pmatrix}$$

$$A_{ii} = 0 \quad A_{ij} = A_{ji}$$
$$E = \frac{1}{2} \sum_{i,j=1}^N \text{nonzero}(A_{ij}) \quad \bar{k} = \frac{2E}{N}$$

Clase 2.1

NetworkX

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl